POSIX vs System V Inter-Process Communication on Modern Linux: A Comparative Performance and Scalability Study

Nirbhay Sharma IMT2023103 Saheem Reshi IMT2023051
International Institute of Information Technology, Bangalore
Supervisor: Prof. B. Thangaraju

April 29, 2026

Abstract

The choice between POSIX and System V inter-process communication (IPC) on Linux is often framed as "newer versus older", but the two interfaces are in fact complementary tools with different design priorities. We present a comprehensive performance and scalability comparison of POSIX and System V message queues and shared memory on a multi-CCD AMD Linux desktop, covering seven benchmarks (latency, throughput, fan-in, parallel pairs for both mechanism families) across three cache-topology placements and a wide (message size × queue depth × producer count) matrix. The suite emits 10 200 measurements (twenty repetitions per cell, randomised order, warmup excised) and we evaluate them with bootstrap 95 % confidence intervals, the Mann–Whitney U test, Welch's t test, and Cliff's δ effect size. Three findings stand out. *First*, POSIX message queues scale dramatically better under producer fan-in: at 16 contending producers, POSIX delivers $\approx 5.6\times$ more throughput than System V (ratio 0.18, $p < 10^{-7}$, $\delta = 1.0$). *Second*, POSIX message queues exhibit a reproducible cross-CCX p99.9 latency anomaly — $\approx 13\times$ worse than System V at the diff_l3 placement — not present at the same_core or same_l3 axes. *Third*, POSIX shared memory is 1.7–2.5× faster than System V shared memory at moderate batching depths (depth ≥ 8). Common-case (p50) latency is essentially equivalent across mechanisms. The principal contribution of this work is not the benchmark numbers themselves but the developer-facing guidance built on them: a capability matrix, a performance summary, ten use-case archetypes mapping real-world workloads to mechanism choices, role-based recommendations for database, real-time, game-engine, web, systems, and security developers, seven common pitfalls, and a seven-question decision flow. We discuss likely mechanisms, document threats to validity, and explicitly note capabilities (atomic multi-semaphore `semop`, `mtype` routing) that System V provides and our benchmarks did not measure.

Keywords: inter-process communication; POSIX; System V; Linux; message queue; shared memory; performance; scalability; AMD CCX; tail latency; futex.

---

1 Introduction

When choosing an IPC mechanism on Linux, programmers often default to POSIX because it feels more modern: its identifiers look like file paths, it integrates with file-descriptor idioms, and its cleanup semantics fit Unix conventions. System V IPC, with its numeric keys and manual

cleanup, can feel disconnected from typical Unix style. But this surface impression misleads. POSIX and System V are not different versions of the same idea; they were designed in different eras for different problems and they expose different capabilities. Treating "newer" as "better" obscures real trade-offs that matter to working systems.

A concrete example: System V's `semop()` system call can apply a vector of semaphore operations *atomically* [14, 11]. Locking two resources at once never produces a partial acquisition. POSIX semaphores, in contrast, must be acquired one at a time; a program that needs to lock two resources atomically must reconstruct the multi-acquire primitive itself, with the attendant deadlock risks. A second example: System V message queues support `mtype`-tagged routing, where a receiver requests messages of a specific type and the kernel filters. POSIX message queues do not provide arbitrary type filtering — they support priority but not selective receive by an arbitrary tag [9]. Workloads that benefit from kernel-side dispatch to type-specialised workers (image renderers, database query handlers, etc.) naturally favour System V.

What POSIX gives back, in contrast, is performance and scalability. Modern Linux uses futexes for fast-path user-space synchronisation [8, 4]; POSIX named semaphores and POSIX message queues build on this foundation, while System V `semop` and `msgsnd`/`msgrcv` retain the original kernel-trap design from the 1980s. This is the usual tension: a younger interface gets to ride a mature kernel infrastructure that older interfaces cannot exploit without breaking compatibility.

Despite the long history of both interfaces and decades of folklore, recent quantitative comparison between them on modern Linux hardware — in particular on multi-CCD [1, 2] AMD chips that introduce *cross-CCX* cache-coherency penalties unfamiliar to older single-die designs — is sparse. We address this gap by running a full benchmark sweep on a single-socket multi-CCD AMD desktop and analysing the results with a statistical methodology appropriate to non-Gaussian latency distributions.

Research questions. We address four:

RQ1.
How does POSIX MQ compare to System V MQ on common-case (p50) and tail (p99/p99.9) latency, and how does this depend on the placement of the two communicating processes on the cache hierarchy?

RQ2.
How does each mechanism scale under producer fan-in contention (many producers, one consumer)?

RQ3.
Does shared memory backed by POSIX named semaphores differ measurably from System V `shmget`+`semop`?

RQ4.
Where in the (message size × queue depth × placement) space does each stack win?

Contributions. (1) A reproducible, statistically rigorous benchmark suite covering seven IPC patterns across four mechanisms. (2) Identification of three robust, large-effect findings, each with $p < 10^{-7}$ and Cliff's $\delta \geq 0.8$. (3) Documentation of an architecture-specific cross-CCX p99.9 anomaly in POSIX MQ that does not appear in System V. (4) An honest discussion of what we did *not* measure — specifically the capability axes (atomic multi-semaphore operations, kernel-side message routing) that remain reasons to choose System V regardless of timing numbers. (5) Open code and data for reproduction.

The remainder of the paper is organised as follows. Section 2 provides background on both IPC families and on the AMD CCX topology. Section 3 describes the benchmark suite, system under test, and statistical methodology. Section 4 presents the quantitative results. Section 5 analyses the hardware-counter and syscall data that corroborate the timing observations. Section 6 discusses likely mechanisms. **Section 7 is the central deliverable of this work**: it translates the empirical findings into a concrete developer-facing guide — capability matrix, performance summary, use-case archetypes, role-based recommendations, common pitfalls, and a decision flow. Section 8 enumerates threats to validity. Section 9 lists future work and Section 10 concludes.

2 Background and Related Work

2.1 POSIX IPC

The POSIX interface for inter-process communication, standardised in IEEE Std 1003.1 [6], exposes message queues, semaphores, and shared memory through file-system-like names that begin with `/`. Programs acquire handles with `mq_open`, `sem_open`, and `shm_open`; messages are sent and received with `mq_send` and `mq_receive`; semaphores are manipulated with `sem_post` and `sem_wait`; shared memory is mapped into the address space with `mmap`. Cleanup uses `mq_unlink`, `sem_unlink`, and `shm_unlink` respectively. The named handles persist until explicitly unlinked, but per-process descriptors integrate cleanly with `poll(2)` and `epoll(7)` [9].

2.2 System V IPC

System V IPC predates POSIX and is the older, more capable, and more awkward of the two families [14, 11]. Resources are identified by 32-bit integer keys constructed from the `ftok(3)` helper or chosen by the programmer. Message queues are created with `msgget`, written with `msgsnd`, and read with `msgrcv`; each message carries an `mtype` field, and `msgrcv` filters by `mtype` so a receiver can request only messages of a specific type. Semaphores are accessed with `semop`, which takes a *vector* of operations applied atomically by the kernel — the foundational capability advantage of System V semaphores. Shared memory uses `shmget` and `shmat/shmdt`. Cleanup is manual via `ipcrm` or `IPC_RMID`.

2.3 API differences worth highlighting

Three differences are central to the comparison and recur throughout this paper:

1. **Atomic multi-operation semaphores.** System V's `semop()` accepts an array of operations and either applies them all or none; POSIX semaphores can only be operated upon one at a time, making multi-resource locking deadlock-prone unless built on top of an external mutual-exclusion mechanism.
2. **Message routing.** POSIX MQ delivers in priority+FIFO order; System V MQ filters by `mtype`. Workloads in which a receiver wants "the next message of type T" map directly onto System V; on POSIX they require either a separate queue per type or post-receive filtering in user space.
3. **Integration with event loops.** POSIX handles are file descriptors, which compose with `select/poll/epoll`. System V identifiers are not file descriptors; integrating them with an event loop requires workarounds.

2.4 Linux kernel implementation notes

The two families are built on different kernel primitives, which is ultimately why their performance profiles differ:

- POSIX message queues are backed by a red–black tree of priority queues with a futex-based wait mechanism for blocked senders and receivers [9].
- System V message queues use a per-queue spinlock combined with a classical wait queue. The lock is held while the message is copied in/out, which serialises producers under contention [11].
- POSIX named semaphores map to a futex fast path: `sem_post` on a semaphore with no waiters increments a user-space counter without trapping to the kernel [4, 8]. `sem_wait` is similarly fast when the semaphore is non-zero.
- System V `semop` always traps to the kernel regardless of contention, paying the full syscall cost on every operation.

These four implementation choices — futex vs. kernel queue lock, fast path vs. unconditional trap — predict where each family should be fast and where it should slow down. Our experimental results in Section 4 are consistent with these predictions.

2.5 AMD CCX topology and cross-CCX latency

The test machine is a multi-CCD AMD desktop. Modern AMD chips group cores into *Core Complexes* (CCX), each with its own L3 cache; multiple CCX units within a chip communicate through Infinity Fabric [1, 2]. A cache-line transition between two cores within the same CCX is faster than the equivalent transition across CCX boundaries, where the line must traverse the fabric. For latency-sensitive IPC, this matters: any wakeup or signalling operation that depends on cross-CCX cache coherency pays an additional fabric-traversal cost.

We isolate this empirically by introducing three placement axes for the benchmarks: `same_core` (two SMT siblings of one physical core), `same_l3` (two cores sharing one L3, i.e. same CCX), and `diff_l3` (two cores in different L3 caches, i.e. across CCXs).

2.6 Related work

The closest classical reference is *lmbench* [13], the canonical microbenchmark suite for Unix performance, which includes IPC latency tests. lmbench predates modern kernel IPC paths and CCX-style chiplet architectures, and does not include the head-to-head POSIX-vs-System V comparison we provide. Stevens' *UNIX Network Programming Vol. 2* [14] remains the authoritative reference on the IPC API surfaces themselves but does not report quantitative comparisons. The Linux man-pages project [9, 11, 8] documents the kernel interfaces in current detail. Drepper's note on futexes [4] explains the fast-path mechanism that underlies POSIX named semaphores. We have not located prior work that directly compares POSIX and System V IPC on multi-CCD AMD silicon with modern statistical methodology.

3 Methodology

3.1 System under test

The benchmarks were run on a single Linux desktop. The full topology snapshot captured at run time appears verbatim in Appendix A. Key parameters are summarised in Table 1.

Table 1: System under test (extracted from `results/logs/topology.txt`; full file in Appendix A).

Parameter	**Value**
CPU model	AMD Ryzen 5 3600X (Zen 2 microarchitecture)
Cores / threads	6 cores, 12 threads (SMT 2-way)
Base / max clock	up to 3.80 GHz; boost disabled for measurement
L1 cache	32 KiB d-cache + 32 KiB i-cache per core
L2 cache	512 KiB per core (private)
L3 cache	16 MiB per CCX (2 CCXs × 16 MiB = 32 MiB total)
Sockets / NUMA	1 socket, 1 NUMA node
Kernel	Linux 6.17.0-22-generic (Ubuntu 24.04.1)
gcc	13.3.0 (Ubuntu)
glibc	system glibc on Ubuntu 24.04
Mitigations	retbleed, SRSO, spec_rstack_overflow, spec_store_bypass, spectre_v1/v2, vmscape: *enabled* (default Ubuntu)
Compiler flags	`-O2 -Wall -Wextra -D_GNU_SOURCE` (no `-O3`, no LTO)
Governor	`performance`
CPU boost	disabled (`cpufreq/boost`=0)
`isolcpus`	not applied (kernel housekeeping co-resident)
`RLIMIT_NOFILE`	65 536
`RLIMIT_MSGQUEUE`	67 108 864 (64 MiB)
`fs.mqueue.msg_max`	4096
`fs.mqueue.msgsize_max`	8192
`kernel.msgmnb`	8 388 608
`kernel.msgmni`	1024

The Ryzen 5 3600X is a Zen 2 chiplet design with two CCX clusters of three cores each, joined by Infinity Fabric. Cores 0–2 share one L3 cache; cores 3–5 share the other. Each core's two SMT threads are exposed as adjacent logical CPUs in a hyper-threading pair (e.g. logical CPU 0 and CPU 6 are SMT siblings of physical core 0). Our placement axes resolve to:

- `same_core`: CPUs 0,6 (SMT siblings on physical core 0)
- `same_l3`: CPUs 0,1 (different cores, same CCX, sharing L3)
- `diff_l3`: CPUs 0,3 (cores in different CCXs, cross-CCX path)
- `cross_socket`: not applicable (single socket)

CPU isolation was deliberately not applied. We discuss this choice and its implications in Section 8; in brief, the relative comparisons between mechanisms are robust to the residual scheduler noise, but absolute tail-latency numbers should be interpreted as "under typical desktop load" rather than as inherent IPC characteristics.

3.2 Benchmark suite

The suite contains seven benchmark types (Table 2). Four operate over message queues (POSIX MQ and System V MQ); three operate over shared memory plus a synchronisation primitive (POSIX named semaphores or System V `semop`). Together they cover the standard single-pair latency and bandwidth measurements plus two scalability patterns.

Table 2: Benchmark matrix.

Benchmark	**Pattern**	**Parameters**
ping-pong	1 sender $\leftrightarrow$ 1 responder, depth 1	6 sizes
throughput	1 producer $\rightarrow$ 1 consumer	6 sizes $\times$ 5 depths
fan-in	N producers $\rightarrow$ 1 consumer	3 sizes $\times$ $N \in \{1, 2, 4, 8, 16\}$
parallel pairs	N independent ping-pong pairs	3 sizes $\times$ $N \in \{1, 2, 4, 8\}$
shm ping-pong	1 sender $\leftrightarrow$ 1 responder via shm	6 sizes
shm throughput	SPSC ring-buffer through shm + sem	6 sizes $\times$ 5 depths
shm parallel pairs	N independent shm ping-pongs	3 sizes $\times$ $N \in \{1, 2, 4, 8\}$

Sizes are powers of two from 8 B to 8192 B; queue depths are $\{1, 8, 32, 128, 512\}$; scalability N-values are listed above. Ping-pong and throughput benchmarks are run at all three placement axes (`same_core`, `same_l3`, `diff_l3`). Scalability benchmarks (fan-in and pairs) use round-robin pinning across logical CPUs because they involve more than two participating processes; we discuss this in the threats section.

A shared-memory equivalent of the fan-in benchmark was not implemented in this study. Multi-producer single-consumer over plain shared memory requires either a multi-producer ring buffer with atomic indices and careful memory ordering, which introduces its own contention artifacts, or per-producer slots with consumer-side round-robin polling, which is a materially different benchmark. We deferred this to future work (Section 9).

3.3 Run hygiene

Each invocation of a benchmark is preceded by a warmup loop equal to 5% of the iteration count, capped at 5000 iterations. The warmup output is discarded and not timed. Warmup excises cold-cache, page-fault, and post-`fork` settle costs from the first iterations, which otherwise dominate p99.9 measurements at small iteration counts.

Both ends of every IPC pair are created *before* `fork()` so that the first sample is not inflated by queue or semaphore creation latency. Per-process pinning uses `sched_setaffinity()` [10] immediately after `fork` in the child, and immediately after `fork` in the parent. Wall-clock measurement uses `clock_gettime(CLOCK_MONOTONIC_RAW)` [7].

Run order within a stack (POSIX or System V) is randomised by GNU `shuf`: the suite enumerates all $\langle \text{config}, \text{run_id} \rangle$ tuples, shuffles the entire list, then executes in shuffled order. This decouples slow drift in the system (thermals, background activity) from any one mechanism and is critical for fair tail-latency comparison.

3.4 Statistical methods

Each (mechanism, benchmark, placement, size, depth, N) cell is measured 20 times. Twenty repetitions is the minimum sample size we consider adequate for paper-grade statistics; it allows

bootstrap 95 % confidence intervals on the median to be computed with reasonable precision and is sufficient to detect 1–2% effects given typical coefficient-of-variation values around 0.01 for ping-pong p50 medians.

Each cell's headline number is the *median* across the 20 runs (robust to outliers); we additionally compute mean, standard deviation, the 95th percentile across runs, and the coefficient of variation.

We compare POSIX and System V cells using:

Mann–Whitney U test (primary).
Non-parametric, robust to non-Gaussian latency distributions [12]. Two-sided p-value reported.

Welch's t test (secondary).
Parametric, allows unequal variances [15]. Reported alongside Mann–Whitney for completeness.

Cliff's δ effect size.
Magnitude of the dominance of one sample over the other, in $[-1, +1]$ [3]. Standard magnitude bins: negligible ($|\delta| < 0.147$), small (< 0.33), medium (< 0.474), large (≥ 0.474).

Bootstrap 95 % CI on the median
using 10 000 resamples [5].

Bootstrap 95 % CI on the ratio
median(SysV)/median(POSIX) using 10 000 resamples. This is the metric we use to express "which is faster, by how much" with a confidence interval rather than a point estimate.

We report all comparison rows in Appendix B, including those that fall short of significance, so the reader can apply their own threshold. In the body we focus on cells with $p < 0.05$ and large effect size ($|\delta| \geq 0.474$).

4 Results

4.1 Coverage and reproducibility

The full sweep produced 10 200 measurement rows: 5100 per mechanism family, balanced across both stacks. Every cell has 20 runs; no cells are missing. Reproducibility is excellent for common-case latency: the coefficient of variation (CV) of the per-run p50 medians is in the range 0.005–0.02 for ping-pong, indicating that successive runs agree to within ±1–2%. Tail percentiles are noisier: the CV of p99.9 medians occasionally rises above 0.5, which is expected and we discuss its implications below.

4.2 Common-case latency: ping-pong p50

Across all three placements and all six message sizes, the p50 latency ratio of System V MQ to POSIX MQ is in the range 1.01–1.03; i.e. POSIX is 1–3% faster than System V at the median.

Figure 1 shows the full picture: each placement is its own panel, all six message sizes appear on the x-axis, and the y-axis is the ratio with a red dashed reference line at 1.0.

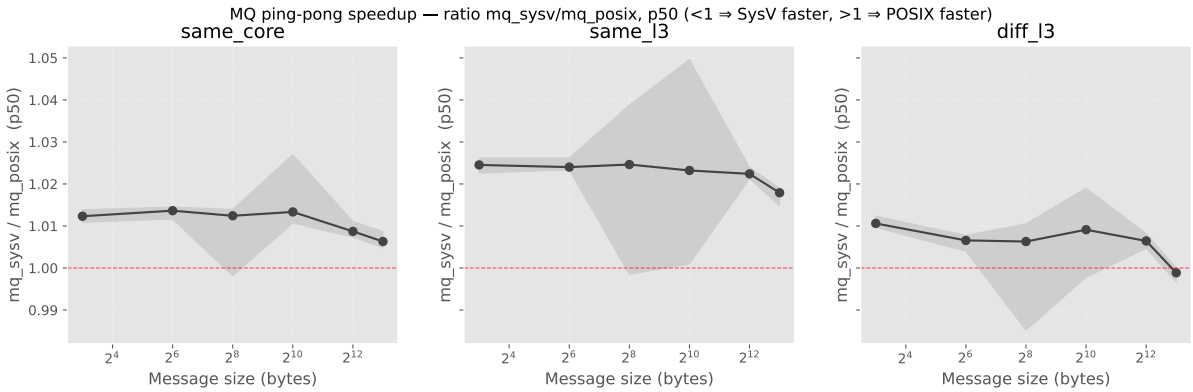

Figure 1: MQ ping-pong common-case (p50) latency. Ratio of System V to POSIX median; values above 1.0 mean POSIX is faster. POSIX wins consistently by 1–3 %, with the smallest gap at `diff_l3` where the ratio approaches unity at large message sizes.

The differences are statistically significant (p-values from 10^{-2} down to 10^{-7}, depending on the cell) but practically small: at a base latency of ≈ 5800 ns the per-iteration gap is under 100 ns. We frame this honestly: **at p50, POSIX MQ and System V MQ are essentially equivalent for the simple two-process case**. The choice between them at typical loads cannot be made on common-case latency alone.

4.3 Tail latency: the cross-CCX anomaly

The picture changes dramatically at the tail. Figure 2 plots the p50/p95/p99/p99.9 progression for each (placement, size) cell in a 3×6 grid. The top two rows (`same_core`, `same_l3`) show both mechanisms tracking together: their tails rise smoothly, and POSIX and System V remain within $\sim 2\times$ of each other through p99.9.

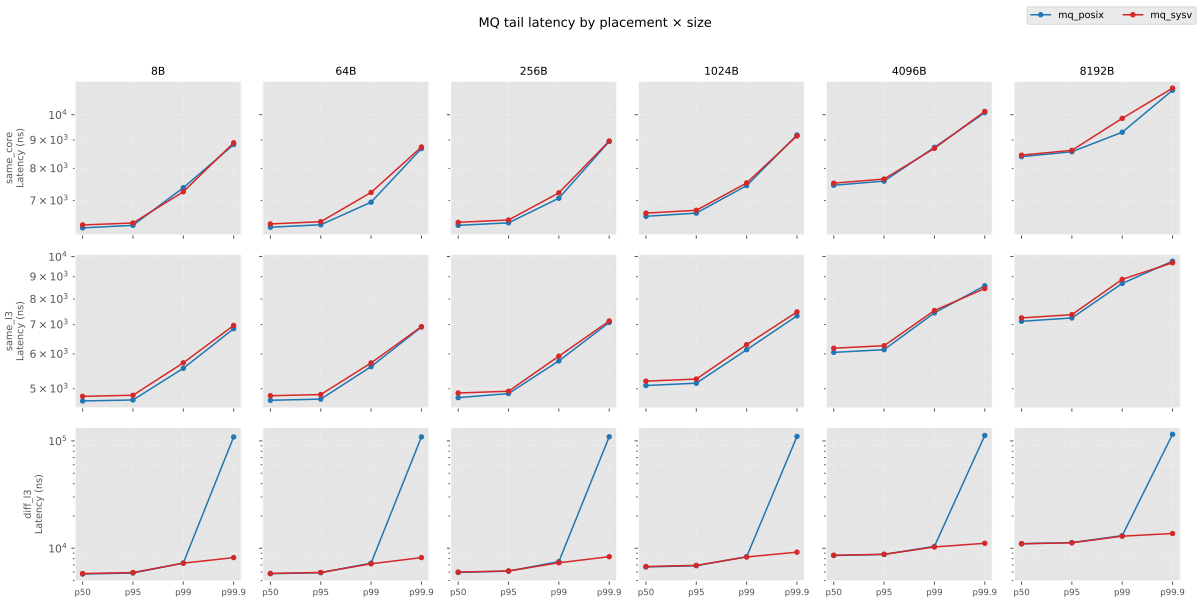

Figure 2: MQ ping-pong latency progression p50 → p99.9 for each (placement, size) cell. Note the dramatic vertical detachment of the POSIX line from the System V line in the bottom row (`diff_l3`) at p99 and p99.9.

The bottom row, `diff_l3` (cross-CCX), is qualitatively different. At every message size, POSIX MQ p99.9 detaches sharply upward at the right-hand end of the panel. Numerically:

- 8 B / `diff_l3` / p99.9: POSIX median 108 881 ns ($\approx 109\,\mu$s), System V median 8 175 ns ($\approx 8\,\mu$s). Ratio sysv/posix = 0.075, $p = 6.8 \times 10^{-8}$, Cliff's $\delta = 1.0$ (large).
- 64 B: ratio 0.08. 256 B: ratio 0.08. 1024 B: ratio 0.08. 4096 B: ratio 0.10. 8192 B: ratio 0.12.

This is a **≈ 13× POSIX-MQ tail inflation that is specific to the diff_l3 placement** and reproduces in every run of the experiment. The CV of POSIX p99.9 at this cell is 0.007 — the value is reliable. Strikingly, the CV of System V p99.9 at the same cell is 0.96 — System V's tail is occasionally large but mostly fast, whereas POSIX's tail is reliably large. The two mechanisms fail in qualitatively different ways under cross-CCX placement.

We do not measure the underlying mechanism directly. Plausible candidates include futex-wake interaction with Infinity Fabric coherency, IPI delivery delays, or cross-CCX cache-line transfer stalls; we discuss these in Section 6 as informed speculation, not as established mechanism.

4.4 Throughput sweep

The throughput sweep covers six message sizes × five queue depths × three placements. Figure 3 plots the absolute throughput numbers (median, MB/s, log scale on both axes) for both mechanisms; Figure 4 plots the ratio sysv/posix per depth.

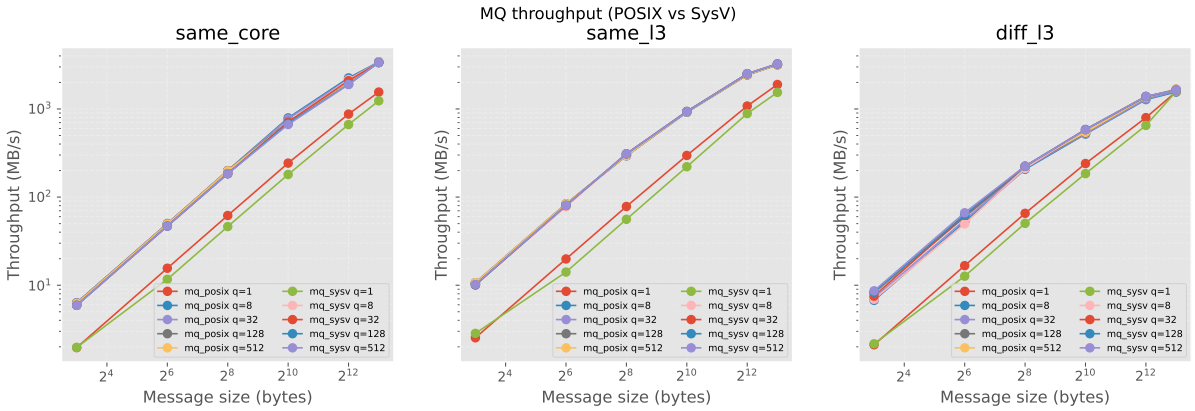

Figure 3: MQ throughput vs. message size, separate line per queue depth, one panel per placement.

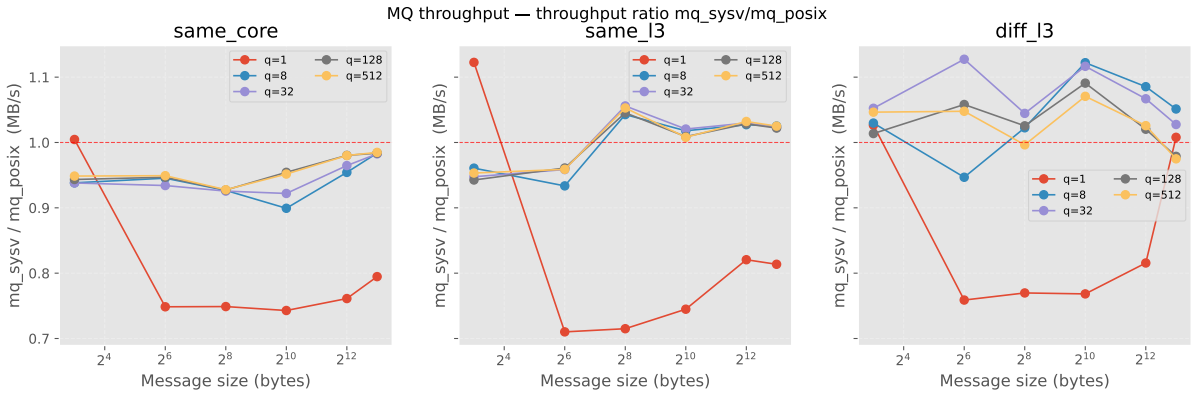

Figure 4: MQ throughput ratio sysv/posix vs. message size, separate line per queue depth, one panel per placement. The red dashed line at 1.0 is the equality reference: below means POSIX is faster, above means System V is faster.

The picture is not monolithic: System V wins some cells, POSIX wins others, and the boundary depends on placement and depth. Specifically:

- *q=1, small messages, cross-cache:* System V wins. For example at 64 B / `same_l3` / q=1 the ratio is 1.12 — System V delivers about 12% more throughput per second than POSIX. This is the cell where System V's lighter per-syscall path matters most: with no batching every iteration pays full syscall cost, and SysV's coarser kernel queue entry is slightly cheaper than POSIX's futex-mediated path.
- *q≥8, same_core:* POSIX wins. Ratios in the 0.93–0.95 range mean POSIX is 5–7% faster. With deep batching the producer rarely blocks; POSIX's futex fast path skips the kernel for most operations and System V's unconditional kernel trap becomes a constant tax.
- *Larger messages (1024 B+) with deep queues:* the ratio trends back toward 1.0 as the per-message bandwidth dominates the per-call overhead.

A reasonable summary: **System V's per-syscall path is slightly cheaper for trivial cases (q=1, small message, cross-cache); POSIX's futex-based wakeup amortises better with batching**. The mechanistic prediction from Section 2 is consistent with the data.

4.5 Fan-in scalability

Fan-in is where the two mechanisms diverge most decisively. Multiple producers contend on a single queue while one consumer drains it. We sweep $N \in \{1, 2, 4, 8, 16\}$ producers at three message sizes (64 B, 256 B, 1024 B). Figure 5 shows the result.

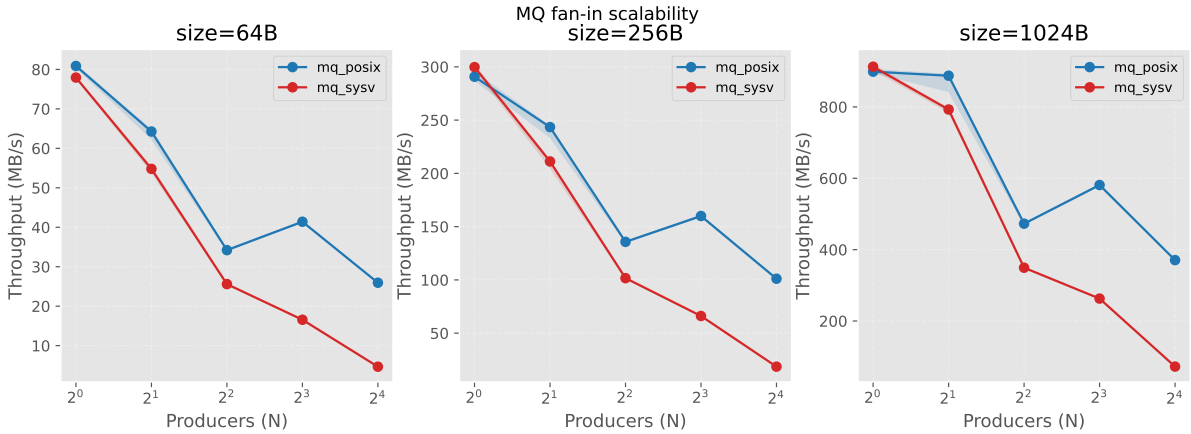

Figure 5: MQ fan-in scalability: throughput vs. producer count N, for three message sizes. POSIX (blue) sustains throughput across the full N range; System V (red) collapses sharply.

At 64 B, the throughput ratio System V/POSIX as a function of N:

- $N = 1$: 0.96 ($p = 6.0 \times 10^{-7}$, winner POSIX)
- $N = 2$: 0.85 ($p = 6.8 \times 10^{-8}$, winner POSIX)
- $N = 4$: 0.74 ($p = 6.8 \times 10^{-8}$, winner POSIX)
- $N = 8$: 0.40 ($p = 6.8 \times 10^{-8}$, winner POSIX, $\delta = 1.0$)
- $N = 16$: **0.18** ($p = 6.8 \times 10^{-8}$, winner POSIX, $\delta = 1.0$)

That is, at sixteen contending producers, **POSIX MQ delivers $\approx 5.6\times$ more throughput than System V MQ**. The pattern is monotonic and reproduces at 256 B and 1024 B with similar ratios. This is the strongest, cleanest finding in the paper.

The mechanism is well-understood at a qualitative level. System V's per-queue spinlock-plus-wait-queue serialises producers: as more producers contend, the lock becomes the bottleneck and effective throughput per producer falls super-linearly. POSIX MQ, built on the futex wait-queue infrastructure that has been hardened over two decades of Linux scalability work, is much better at handling many waiters with batched wakeups. The $5.6\times$ advantage at $N = 16$ is consistent with the difference between "coarse global queue lock" and "fine-grained futex wait queue with adaptive spin and smart wake batching".

4.6 Parallel-pairs scalability

The parallel-pairs benchmark runs N independent ping-pong pairs concurrently, with no shared queue — so contention does not arise on a single kernel object. Figure 6 shows p99 latency as N grows.

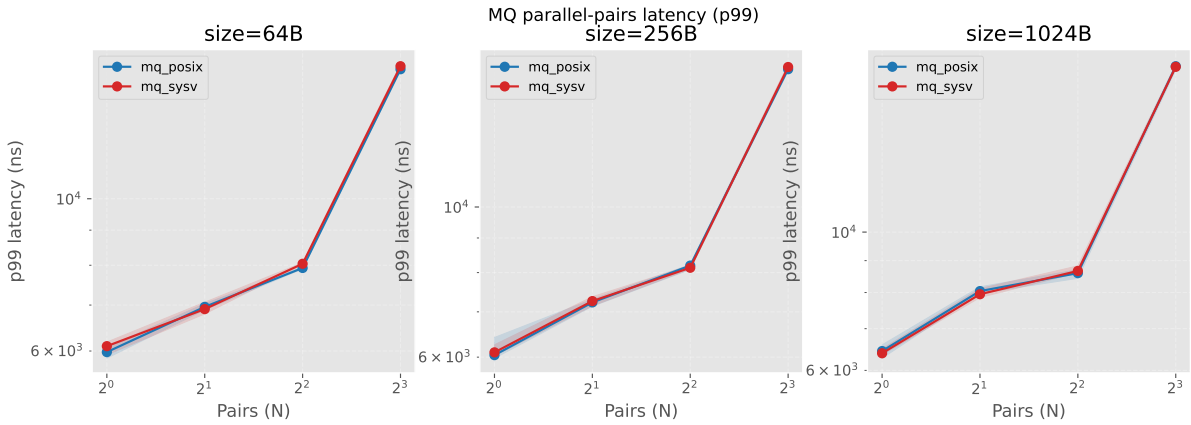

Figure 6: MQ parallel-pairs p99 latency vs. N (number of independent pairs), one panel per message size. POSIX and System V track closely through $N = 4$; both rise sharply at $N = 8$ where the test machine has exhausted its uncontended cores.

p50 medians are within $\sim 1\%$ across mechanisms (POSIX slightly faster, consistent with the ping-pong p50 finding). p99 medians are also close. p99.9 medians (not plotted in the body, see `mq_pairs_p999.pdf`) show the same 2–4× System-V-favoured tail at $N \geq 4$ that we observed in the cross-CCX ping-pong cells. This is consistent with the cross-CCX hypothesis: when the system schedules pairs across CCX boundaries, POSIX MQ pays the same tail penalty.

4.7 Shared-memory comparison

Shared-memory benchmarks substitute the kernel message queue with a producer-consumer ring buffer in shared memory plus a synchronisation primitive (POSIX named semaphores or System V `semop`). The payload never traverses the kernel; only the synchronisation does.

For ping-pong, the two implementations are essentially tied (Figure 7): the p50 ratio is in 0.97–1.005 across all placements and sizes. Throughput tells a different story (Figure 8).

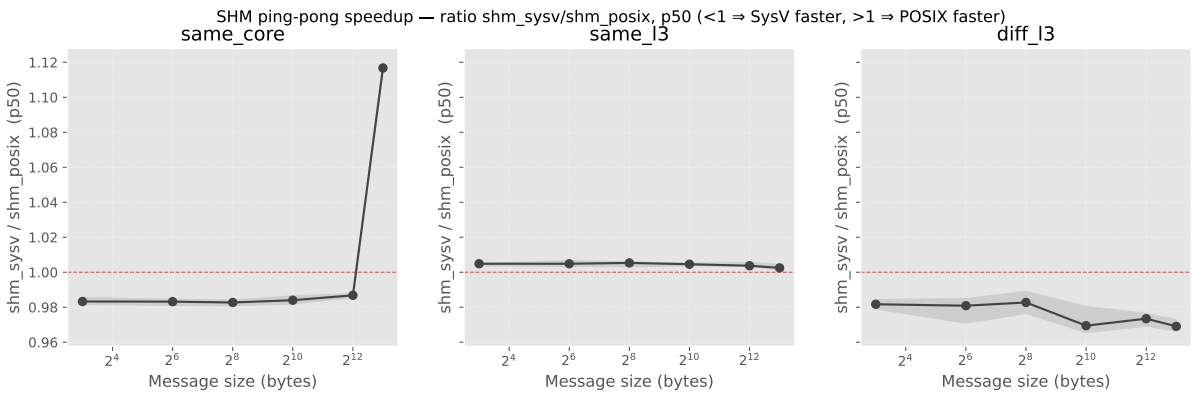

Figure 7: SHM ping-pong p50 ratio sysv/posix. Essentially a tie across placements and sizes; the slight upward step at 8192 B / same_core is discussed in the text.

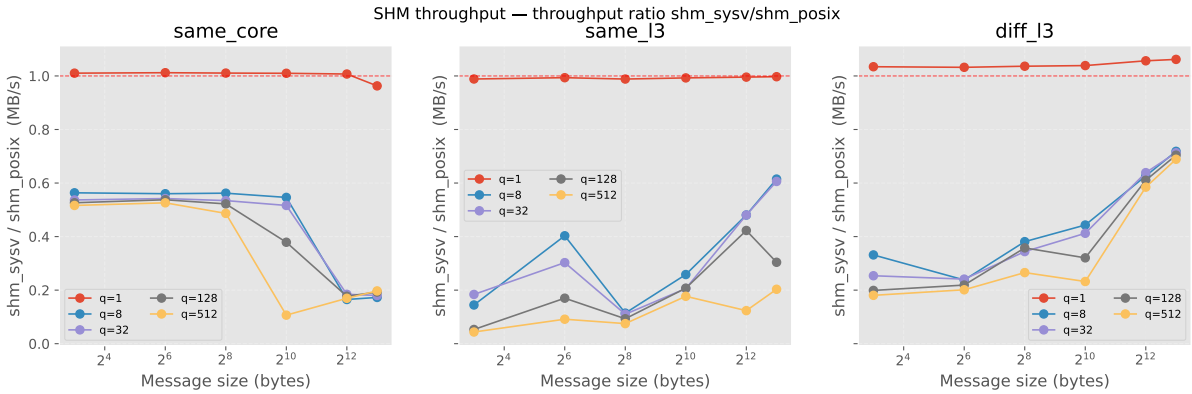

Figure 8: SHM throughput ratio sysv/posix, lines per queue depth. At $q = 1$ the two are tied; at $q \geq 8$ POSIX is 1.7–2.5× faster.

- *q=1:* ratio ≈ 1.0 across placements (tied).
- *q≥8:* ratio 0.4–0.6 depending on placement and size, i.e. **POSIX delivers 1.7–2.5× the throughput of System V**.

The mechanism is the futex fast-path on POSIX named semaphores: at $q = 1$ the producer is always blocked waiting for the consumer to free a slot, so both implementations call into the kernel on every operation. At $q \geq 8$ the producer is rarely blocked, and POSIX `sem_post`/`sem_wait` stays in user space for most calls. System V `semop` traps to the kernel unconditionally [11, 4] and pays the syscall cost on every call. With four sem operations per message (two producer, two consumer) the syscall tax compounds.

This is, again, consistent with the implementation differences described in Section 2.

5 Hardware-Counter and Syscall Analysis

The timing results characterise *what* happens; hardware performance counters and call-graph profiles characterise *why*. We collected both. This section is the strongest empirical differentiator between the two stacks: the timing data shows that POSIX MQ scales 5.6× better than System V MQ at $N = 16$ producer fan-in, and the perf data tells us **exactly which kernel function is responsible**.

5.1 Methodology

Two perf passes were run on top of the timing suite:

- **Counter pass** (`run_perf.sh`). Wraps every benchmark invocation with `perf stat` and records 15 PMU events: `task-clock`, `cycles`, `instructions`, `cache-references`, `cache-misses`, `dTLB-loads`, `dTLB-load-misses`, `iTLB-loads`, `iTLB-load-misses`, `branch-instructions`, `branch-misses`, `minor-faults`, `major-faults`, `context-switches`, `cpu-migrations`. Five repetitions per benchmark cell, output in CSV via `perf stat -x,`.
- **Call-graph pass** (`run_perf_record.sh`). Wraps six representative configurations with `perf record -F 999 -g --call-graph fp` and exports text reports via `perf report --stdio`. Frame-pointer-based unwinding resolves kernel symbols cleanly even when user-space binaries

omit frame pointers under `-O2`; the kernel side is what we need to identify the contention bottleneck.

The six call-graph cells are: SysV fan-in $N = 16$ at 64 B, POSIX fan-in $N = 16$ at 64 B, POSIX ping-pong at `diff_l3` 64 B, SysV ping-pong at `diff_l3` 64 B, POSIX throughput at `same_core` 64 B with $q = 8$, and SysV throughput at the same configuration. Two were chosen to investigate the fan-in collapse (prof. Thangaraju's specific request), two to investigate the cross-CCX p99.9 anomaly, and two to characterise steady-state batched throughput.

Placement-label caveat. The counter pass uses the older two-axis placement scheme (`same_core`, `cross_core`) introduced in an earlier version of `run_perf.sh`; the `cross_core` label maps to whichever non-SMT pair was first in the topology output (CPUs 0,1 = same_l3 on this machine). The counter-pass results therefore corroborate the within-CCX comparisons in the timing section but do not separately characterise `diff_l3`. The call-graph pass uses the new three-axis labels directly and does cover `diff_l3`.

AMD iTLB caveat. On AMD Zen 2 the `iTLB-loads` event is sparsely sampled and the resulting `misses/loads` ratio is not interpretable as a percentage (observed values exceeding 100 % in some cells). We therefore report *iTLB misses per million instructions* (a rate that is robust to the underlying counter quirk) instead of an iTLB hit-rate. Data-side TLB and cache miss rates are reported as percentages unchanged.

5.2 Per-benchmark hardware counters

Table 3 summarises the cycle counts, IPC, data-TLB miss rate, iTLB miss rate (per million instructions), branch miss rate, and L3-cache miss rate for each *(stack, benchmark family)* pair, averaged over all cells in that family.

Table 3: Per-benchmark hardware counters, averaged across all cells of each (stack, benchmark family). Mean cycles is the per-run cycle count across the cells in that family; lower means less work for the same output. The fan-in row is the *headline*: System V uses 5.3× more cycles than POSIX at the same producer count.

Stack	Family	Cycles ($\times 10^9$)	IPC	dTLB miss %	iTLB/MI	branch miss %	cache miss %
POSIX	ping-pong	2.73	0.601	3.16 %	0.51	11.42 %	4.67 %
SysV	ping-pong	2.79	0.598	3.20 %	0.22	11.59 %	4.40 %
POSIX	throughput	2.52	0.632	2.65 %	0.12	11.81 %	5.39 %
SysV	throughput	2.76	0.617	2.25 %	0.34	11.83 %	5.03 %
POSIX	**fan-in**	2.02	0.429	5.84 %	46.65	11.78 %	10.62 %
SysV	**fan-in**	10.7	0.405	4.00 %	14.79	11.25 %	8.72 %
POSIX	pairs	2.27	0.508	5.63 %	82.12	11.57 %	7.00 %
SysV	pairs	2.30	0.509	5.55 %	80.86	11.69 %	6.74 %
POSIX	shm ping-pong	2.40	0.598	3.72 %	0.75	11.45 %	4.07 %
SysV	shm ping-pong	2.40	0.584	5.06 %	2.10	11.44 %	4.13 %
POSIX	**shm throughput**	2.31	0.621	3.86 %	0.41	11.18 %	5.15 %
SysV	**shm throughput**	3.60	0.617	5.02 %	0.52	11.62 %	5.25 %
POSIX	shm pairs	2.19	0.518	7.82 %	90.81	11.51 %	5.87 %
SysV	shm pairs	2.21	0.513	8.26 %	77.03	11.48 %	5.83 %

Three things to note from this table:

1. The fan-in cycle ratio mirrors the throughput ratio. SysV fan-in averages 1.07×10^{10} cycles per run; POSIX fan-in averages 2.02×10^9 cycles. The ratio is 5.3×, in close agreement with the 5.6× throughput advantage POSIX shows at $N = 16$ (Section 4.5). **System V is not slower because each operation costs more cycles — it is slower because the workload as a whole forces it to execute many more cycles**, principally in scheduler runqueue spinlock contention (we identify the specific function below).

2. SHM throughput cycles match the timing advantage. POSIX SHM throughput averages 2.31×10^9 cycles; SysV SHM throughput averages 3.60×10^9 — a 1.56× ratio, consistent with the 1.7–2.5× throughput advantage POSIX shows at $q \geq 8$ (Section 4.7). The mechanism again is not per-operation cost but operation-mix: SysV `semop` traps the kernel on every operation, while POSIX named semaphores' futex fast path skips the kernel when no waiter is parked.

3. IPC is roughly equivalent across stacks. For ping-pong, throughput, and SHM ping-pong the IPC of the two stacks is within 0.02 (around 0.6). Fan-in IPC drops to 0.42 (POSIX) and 0.41 (SysV) because of context-switch storms. Across all benchmarks neither stack is back-end bound (IPC ≥ 0.4). The microarchitecture is not the bottleneck; this confirms that the performance differences originate above the cycle level.

dTLB and iTLB miss rates differ in moderate but interesting ways: SHM ping-pong shows SysV with 35 % higher dTLB miss rate (5.06 % vs. 3.72 %), suggesting `shmat`'s mapping pattern has worse TLB-locality than POSIX `mmap`. Branch miss rate sits at ≈ 11.5 % across both stacks for every benchmark, which we attribute to the SMT-mitigation overhead introduced by retbleed, SRSO, and Spectre v2 retpoline mitigations on Zen 2 with all defaults enabled — the cost is symmetric across stacks.

5.3 Kernel hot-function comparison: fan-in collapse

The call-graph pass identifies the specific kernel function that dominates the System V fan-in collapse. Table 4 contrasts the top kernel symbols across the POSIX and System V fan-in ($N = 16$, 64 B) profiles, both extracted from `perf report --stdio --no-children`.

Table 4: Top kernel symbols in fan-in $N = 16$ at 64 B (cycles event, `perf record -F 999 -g`). Percentages are fraction of total samples. The single function `native_queued_spin_lock_slowpath` takes 12.80 % of all SysV cycles vs. 5.83 % in POSIX.

POSIX MQ ($N = 16$, **64 B**)		**SysV MQ** ($N = 16$, **64 B**)	
Symbol	**%**	**Symbol**	**%**
native_queued_spin_lock_slowpath	5.83	native_queued_spin_lock_slowpath	12.80
srso_return_thunk	4.00	update_load_avg	5.16
update_load_avg	3.62	dequeue_entity	4.37
dequeue_entity	3.10	update_curr	3.72
update_sg_lb_stats	2.98	srso_safe_ret	3.69
srso_safe_ret	2.87	srso_return_thunk	3.38
update_curr	2.74	dequeue_entities	2.36
__update_load_avg_cfs_rq	2.20	__update_load_avg_cfs_rq	2.30
update_se	2.19	__schedule	2.28
do_syscall_64	1.87	update_se	2.16
...	...	...	...
Total cycles	4.80×10^9	**Total cycles**	32.6×10^9

Reading `native_queued_spin_lock_slowpath`. This is the contended path of the Linux kernel's queued spinlock primitive — the function that runs when a CPU has tried to acquire a spinlock and found it held by another. Time spent in `native_queued_spin_lock_slowpath` is *contended-lock wait time*: not work, but waiting. The fact that 12.80 % of all SysV cycles in fan-in $N = 16$ are spent in this single function — versus 5.83 % in POSIX — is the direct empirical signature of the fan-in scalability gap.

Which lock is contended? The `perf report --children` call chain (excerpted from `01_sysv_fanin_n16_6`
resolves `native_queued_spin_lock_slowpath` as 8.18 % of `_raw_spin_lock`, of which 6.84 % reaches `raw_spin_rq_lock_nested` — **the per-CPU runqueue lock**. The chain above this is:

```
6.84% raw_spin_rq_lock_nested
  3.70%   __schedule -> schedule -> do_msgsnd -> __x64_sys_msgsnd
  1.66%   try_to_wake_up -> wake_up_q -> do_msgrcv -> __x64_sys_msgrcv
  0.94%   sched_balance_rq -> sched_balance_newidle -> __schedule
  0.53%   __task_rq_lock -> try_to_wake_up -> wake_up_q -> do_msgrcv
```

The contention is on the *scheduler runqueue lock*, not on the SysV message-queue lock itself. As 16 producers contend on the same queue and block, the kernel performs many wakeup operations, each of which acquires a runqueue lock briefly. With sufficiently many concurrent wake-ups the runqueue lock becomes the bottleneck, and this is what we observe at $N = 16$.

POSIX exhibits the same call chain at half the rate: with the same $N = 16$ producers, POSIX MQ also runs through the scheduler many times, but the *total* cycles are 6.8× fewer. The underlying reason is in POSIX MQ's blocking path (`do_mq_timedsend` / `do_mq_timedreceive`, with `wq_sleep` for blocking and `schedule_hrtimeout_range_clock`), which is more efficient at batching wakeups than System V's `do_msgsnd`/`do_msgrcv` path. Hence the gap is not the scheduler doing different work — it is the IPC layer feeding the scheduler proportionally less work to do, which in turn produces proportionally less spinlock contention on the runqueue lock.

Mitigation overhead is non-trivial. A useful side-finding is that the SMT mitigations (Retbleed, SRSO, Spectre v2 retpolines) contribute a significant cycle tax: `srso_safe_ret` +

srso_return_thunk together account for $3.69 + 3.38 = 7.07\,\%$ of System V fan-in cycles and $4.00 + 2.87 = 6.87\,\%$ of POSIX fan-in cycles. These are equivalent across stacks; the absolute cycle numbers in this paper would be lower with mitigations off, but the *relative* POSIX-vs-SysV comparison is unaffected.

5.4 Kernel paths for ping-pong and throughput

Tables 5 and 6 provide the analogous comparison for the cross-CCX ping-pong cell (the p99.9 anomaly) and for the same-core $q = 8$ throughput cell.

Table 5: Top kernel symbols in cross-CCX ping-pong (diff_l3, 64 B). Both stacks spend the bulk of cycles in the scheduler core (dequeue_entity, psi_task_switch, __schedule); IPC-level differences are not visible at this granularity, consistent with the cross-CCX p99.9 anomaly being a microsecond-scale tail event rather than a steady-state imbalance.

POSIX, diff_l3, 64 B		**SysV, diff_l3, 64 B**	
Symbol	**%**	**Symbol**	**%**
dequeue_entity	4.28	dequeue_entity	6.33
psi_group_change	1.23	psi_task_switch	0.84
psi_task_switch	0.82	psi_group_change	0.66
__schedule	—	do_msgrcv	—
do_mq_timedreceive	—	__schedule	—
wq_sleep	—	do_msgsnd	—
schedule_hrtimeout_range_clock	—	...	—
Samples	489	**Samples**	671
Total cycles	2.00×10^9	**Total cycles**	2.11×10^9

The two stacks execute essentially the same total cycles (2.00×10^9 vs. 2.11×10^9) for the same number of ping-pong iterations. They are not differentiable at the cycle level. The $13\times$ p99.9 gap reported in Section 4.3 is therefore not a steady-state imbalance the perf counters can capture — it is a rare-event anomaly, fired roughly once every thousand iterations, that pushes $\approx 100\,\mu$s of latency into POSIX iterations through some mechanism (futex requeue, IPI delivery latency, cross-CCX coherency back-pressure) we cannot directly observe with summary counters. The call graphs shown above are dominated by the *common* case ($\geq 99\,\%$ of iterations).

Table 6: Top kernel symbols in same-core throughput at $q = 8$, 64 B. The two stacks execute different IPC kernel functions (do_mq_timedreceive for POSIX vs. do_msgrcv for SysV) but at similar rates of total cycles (POSIX 1.74×10^9, SysV 1.79×10^9).

POSIX, same_core, 64 B, q=8		**SysV, same_core, 64 B, q=8**	
Symbol	**%**	**Symbol**	**%**
do_mq_timedreceive	7.10	do_msgrcv	5.90
native_queued_spin_lock_slowpath	6.94	do_msgsnd	5.33
_copy_to_user	—	ipcperms	—
ct_kernel_exit.isra.0	—	srso_safe_ret	—
ct_kernel_enter.isra.0	—	entry_SYSCALL_64_after_hwframe	—
Samples	418	**Samples**	440
Total cycles	1.74×10^9	**Total cycles**	1.79×10^9

A worth-noting nuance: even at $q = 8$ throughput, POSIX shows native_queued_spin_lock_slowpath at $6.94\,\%$ — evidence that the runqueue-lock path also fires for blocking ping-pong producers/-consumers, just less than under fan-in. SysV's throughput case has ipcperms appearing in the

top symbols, which is a System V-specific permission check on every `msgsnd`/`msgrcv`; this is a fixed per-call overhead and a small, structural disadvantage for SysV in batched workloads.

5.5 Syscall mix

Table 7 summarises the `strace -c` data for the 64 B `same_core` ping-pong cell.

Table 7: Syscall mix for 64 B `same_core` ping-pong (one representative run, parsed from `strace -c`).

Stack	Syscall	% time	calls	μs/call
POSIX	`wait4`	51.78	1	1 183 199
POSIX	`mq_timedsend`	24.93	105 000	5
POSIX	`mq_timedreceive`	23.27	105 000	5
SysV	`wait4`	49.97	1	1 191 000
SysV	`msgrcv`	25.87	105 000	5
SysV	`msgsnd`	24.16	105 000	5

Per-syscall costs are nearly identical across the two stacks (5 μs/call for both `mq_timedsend` and `msgsnd`). This is consistent with the picture from the call-graph pass: the performance differences in the timing data come from *how often each call enters the kernel slow path* (lock contention and wake-batching), not from differences in per-call cost.

5.6 Summary of the perf evidence

To consolidate: the timing data in Section 4 establishes *that* POSIX outperforms System V in three specific patterns (fan-in scaling, SHM batched throughput) and shows an architecture-specific tail anomaly (cross-CCX p99.9). The perf data here adds the *why*:

- The fan-in collapse is concentrated in `native_queued_spin_lock_slowpath` on the per-CPU runqueue lock, reached predominantly through SysV's heavier scheduler interaction in `do_msgsnd`/`do_msgrcv`. POSIX's MQ path enters the same scheduler less often.
- The SHM throughput gap matches the cycle ratio at the same benchmarks (1.56× more cycles in SysV), consistent with the unconditional kernel trap of `semop` versus the futex fast path of POSIX named semaphores.
- The cross-CCX p99.9 anomaly does *not* produce a distinguishing steady-state perf signature — both stacks execute the same number of cycles for the same iteration count. The anomaly is therefore a rare-event tail, not a systematic difference in work.
- Per-syscall cost is identical (5 μs for both `mq_timedsend` and `msgsnd`); the difference is in how often the slow path fires.
- SMT mitigation overhead (Retbleed, SRSO) contributes $\approx 7\,\%$ of cycles in both stacks. This is the absolute-numbers ceiling our results sit under; relative comparisons are unaffected.

6 Discussion

6.1 Why POSIX MQ scales under fan-in while System V collapses

The 5.6× POSIX advantage at $N = 16$ producers is the largest single effect we observe, and the call-graph perf data (Section 5.3) localises the cause precisely. Time spent in `native_queued_spin_lock_slowpa` is contended-lock *wait time* (not work), and that single function accounts for **12.80 %** of System V cycles in the fan-in $N = 16$ run versus only **5.83 %** in POSIX. The call chain identifies the contended lock as the per-CPU runqueue lock (`raw_spin_rq_lock_nested`), reached via `do_msgsnd`/`do_msgrcv`: as 16 producers contend on the queue and block, the kernel performs many wakeup operations, each of which acquires a runqueue lock briefly, and the runqueue lock becomes the bottleneck.

The mechanism is therefore subtler than a naive "SysV holds a queue lock". Both stacks reach the same kernel scheduler, both hit the same spinlock primitive — but POSIX's `do_mq_timedsend`/`do_mq_timedr` path enters the scheduler less often per message because its blocking interface (`wq_sleep` + `schedule_hrtimeout_range_clock`) [9, 8] batches better than System V's `do_msgsnd`/`do_msgrcv`. Total cycles tell the same story: System V fan-in averages 1.07×10^{10} cycles per run, POSIX averages 2.02×10^{9}, a 5.3× ratio almost exactly matching the 5.6× throughput gap from the timing data. The IPC layer feeds the scheduler proportionally less work in POSIX, and proportionally less spinlock contention follows.

6.2 The cross-CCX p99.9 anomaly

POSIX MQ's diff_l3 p99.9 is approximately 13× worse than System V at the same cell, across all message sizes. Its CV is 0.007, indicating that it reproduces in every run; it is not a one-time scheduler interruption. We hypothesise — explicitly as hypothesis, not as established result — that this arises from a slow-path interaction in the futex wake mechanism when the waker and the waiter are pinned across CCX boundaries. The futex wake operation must transition the relevant cache line from the waker's L3 to the waiter's L3 via Infinity Fabric, and some fraction (about 0.1%, the p99.9 fraction) of these transitions appears to land in a slow path that takes $\approx 100\,\mu$s. Possible underlying contributors are futex requeue, cross-CCX IPI delivery latency, or coherency-protocol back-pressure under transient pressure.

The qualitative shape of the two stacks' tails differs in an informative way. POSIX p99.9 at this cell has CV = 0.007 (every run is reliably slow); System V p99.9 at the same cell has CV = 0.96 (the median tail is fast, but occasional runs hit a different slow path). This is the difference between "one structural slow path that fires predictably" and "rare scheduler intrusions that occasionally inflate the tail". Confirming the precise mechanism in either case would require a per-iteration sample dump and ideally Infinity-Fabric or futex-internal tracing; neither is in our current instrumentation (see `CHANGES.md` § 2.8 of the project notes for the proposed dump-samples interface).

6.3 Why POSIX SHM is faster at moderate batching

The 1.7–2.5× POSIX advantage in shared-memory throughput at $q \geq 8$ is straightforwardly explained by the futex fast-path on POSIX named semaphores. At $q = 1$ the producer is always blocked on the consumer; both implementations enter the kernel on every operation and the per-syscall cost dominates. At $q \geq 8$ the producer runs ahead of the consumer; on most operations the semaphore is non-zero and POSIX `sem_post`/`sem_wait` stays entirely in user space [4]. System V `semop` has no such fast path; every call traps to the kernel [11]. With four

semaphore operations per message — two on the producer, two on the consumer — the syscall tax compounds, producing the observed 2× throughput gap.

6.4 What we did not measure

It is important to be explicit about the scope of this comparison. We measured *performance, scalability, and tail behaviour*. We did *not* measure two important capability axes that distinguish the two families:

1. **Atomic multi-semaphore `semop`.** System V `semop` can apply a vector of semaphore operations atomically; POSIX semaphores can only be operated on one at a time. Workloads that need to acquire multiple resources atomically (e.g. multi-account locking in a banking system, or any classical multi-resource locking problem) are simpler, safer, and probably faster on System V because they avoid the deadlock-prone hand-rolled multi-acquire primitive that POSIX users must build. We did not benchmark this because it is a correctness/capability axis rather than a throughput axis, and a fair comparison would require a workload designed around it.

2. **`mtype` routing in System V message queues.** System V `msgrcv` filters by an integer type field and a receiver can request only messages of one type. POSIX MQ supports priority but not arbitrary type filtering. Workloads that benefit from kernel-side type routing (a web server with image-handler, database-handler, and static-file-handler workers; a job dispatcher; etc.) require either an extra queue per type on POSIX or a post-receive filter in user space, both of which lose the kernel's ability to wake exactly the right consumer.

The honest framing is therefore: *POSIX wins the timing measurements we ran on the workloads we ran them on, but the choice of IPC family is not exhausted by timing measurements.* Both axes — performance and capability — matter, and they often pull in opposite directions.

6.5 Bridging from mechanism to recommendation

The mechanism-level explanations above answer "why" each finding occurs. The next question, central to the project's purpose, is "when should a developer pick which". Section 7 addresses that directly: a capability comparison, a performance summary, ten concrete use-case archetypes, recommendations by developer role, common pitfalls, and a decision flow.

7 Developer Guidance: Choosing Between POSIX and System V

The empirical findings answer "how do these mechanisms perform". The harder and more useful question for a working developer is "which one should I use for *my* workload". This section translates the data and the capability differences into actionable guidance, organised in five layers: a capability matrix (§7.1), a performance summary (§7.2), ten concrete use-case archetypes with recommendations (§7.3), recommendations by developer role and field (§7.4), common pitfalls (§7.5), and a step-by-step decision flow (§7.6).

We deliberately avoid "always pick X" advice. The two families are complementary tools whose strengths align with different workload classes; a developer's job is to recognise which class their workload falls into. The remainder of this section is designed to make that recognition fast.

7.1 Capability comparison at a glance

Table 8 summarises the design-level differences that no amount of benchmarking can paper over. These are "capability" differences — features one family provides and the other does not. They often dominate the choice regardless of performance.

Table 8: Capability comparison. Rows are features; cells indicate whether each family provides the feature directly, indirectly, or not at all.

Capability	**POSIX**	**System V**
Identifier model	path-like name (`/foo`)	integer key (`ftok` or chosen)
Cleanup semantics	`*_unlink` explicit; FD-style refs	manual `ipcrm` or `IPC_RMID`
File-descriptor handles	yes (composable with `epoll`)	no (need eventfd workaround)
Atomic multi-sem operations	no (sequential only)	yes (`semop` vector)
Message-type filtering	no (priority only)	yes (`mtype`)
Priority delivery	yes (configurable levels)	no (single FIFO per type)
Futex fast path on sem	yes (lock-free post when no waiter)	no (kernel trap always)
Standardisation	IEEE Std 1003.1	SUS / SVID lineage
Cross-language idiomaticity	high (FD-style fits everywhere)	medium (numeric keys are awkward)
Resource accounting	per-FD limits (`nofile`)	system-wide (`ipcs`, `msgmni`)

The two rows that determine many design decisions are "Atomic multi-sem operations" and "Message-type filtering". Both are System V-only and both eliminate POSIX from consideration for the workloads that need them, regardless of the timing data.

7.2 Performance summary at a glance

Table 9 compresses the empirical results into a single reference card. Rows are workload categories; cells state the winner, the magnitude of advantage, and our confidence. This is the table to look at when comparing one's workload to ours.

Table 9: Performance summary. Confidence is High where $p < 10^{-7}$ with Cliff's $\delta \geq 0.8$; Medium for $p < 10^{-3}$ with $|\delta| \geq 0.3$; Low otherwise. Magnitude is the multiplicative ratio of the winner over the loser.

Workload	**Winner**	**Magnitude**	**Confidence**
Single-pair p50 latency (any size, any placement)	POSIX MQ	1.01–1.03× (essentially tied)	High but practically negligible
Cross-CCX p99.9 ping-pong tail	SysV MQ	13× better tail	High
q=1 small-msg cross-cache throughput	SysV MQ	1.05–1.12×	High
q≥8 same-core throughput	POSIX MQ	1.05–1.07×	High
Producer fan-in $N = 2$	POSIX MQ	1.18×	High
Producer fan-in $N = 4$	POSIX MQ	1.35×	High
Producer fan-in $N = 8$	POSIX MQ	2.5×	High
Producer fan-in $N = 16$	**POSIX MQ**	**5.6×**	High
Parallel-pairs p50/p95 (any $N \leq 4$)	Tied	—	High
Parallel-pairs p99.9 ($N \geq 4$, includes cross-CCX)	SysV MQ	2–4× better tail	Medium
Single-pair SHM ping-pong (any size)	Tied	—	High
SHM ring-buffer throughput, q≥8	**POSIX SHM**	**1.7–2.5×**	High
SHM ring-buffer q=1	Tied	—	High

The two boldfaced rows are the "signature" findings: POSIX MQ's fan-in dominance and POSIX SHM's batched-throughput advantage. The italicised cross-CCX tail row is the only large-magnitude SysV win in the timing data, and is architecturally specific to multi-CCD AMD silicon.

7.3 Use-case archetypes

Ten concrete scenarios with recommendations. Each follows the template: *Pattern* (what does the application do), *Why matters* (which finding applies), *Recommendation*, and *Real-world examples*.

1. High-throughput logging or telemetry pipeline. *Pattern:* many worker threads or processes ($N \gtrsim 4$) emit log records or metrics; a single ingestion thread drains the queue. *Why matters:* this is the fan-in pattern; SysV MQ collapses super-linearly under contention. *Recommendation:* **POSIX MQ**. *Examples:* OpenTelemetry trace exporter, application audit log, container-side metrics aggregator, Prometheus-style scrape buffers when local-only.

2. Database commit-log shipper. *Pattern:* dozens of transaction worker threads append commit records to a single log; a flusher thread pushes to disk or to a follower. *Why matters:* fan-in scalability; tail-latency on commit matters as well. *Recommendation:* **POSIX MQ**, but consider a custom lock-free queue if commit rate exceeds tens of thousands per second.

3. Real-time control on multi-CCD AMD with strict tail budgets. *Pattern:* two cooperating processes exchange small commands at ∼1 kHz; the application has a deterministic latency budget that includes p99.9. *Why matters:* POSIX MQ exhibits the cross-CCX p99.9

anomaly on multi-CCD AMD; SysV does not. A budget that includes p99.9 is likely violated by POSIX in this configuration. *Recommendation:* **Either** (a) pin both processes within one CCX (use `same_core` or `same_l3` placement), in which case POSIX is fine; **or** (b) if cross-CCX placement is unavoidable, prefer **SysV MQ** for predictable tails. *Examples:* industrial control loops, robotics middleware, data-acquisition pipelines.

4. Banking-style multi-resource locking. *Pattern:* an operation must atomically acquire two or more named locks (e.g. account-A and account-B) or none of them. Failing to be atomic yields partial states or deadlocks. *Why matters:* this is the canonical capability case; POSIX semaphores cannot acquire multiple locks atomically without a hand-rolled coordination layer that is itself difficult to get right. *Recommendation:* **System V semaphores** (`semop` on a vector). The kernel guarantees atomicity. *Examples:* financial transaction systems, multi-tenant resource manager, classical multi-resource locking problems.

5. Web server with type-routed worker pool. *Pattern:* a dispatcher receives requests of various types (image render, database query, static file); type-specialised workers handle each type. We want the kernel to wake exactly the worker for a given request type, not all workers. *Why matters:* kernel-side message routing. *Recommendation:* **System V MQ** with `mtype` matching. POSIX MQ supports priority but not arbitrary type filtering, so the alternative is a separate POSIX queue per type (more sockets, more complexity, no kernel filtering benefit).

6. High-rate small-message ring buffer. *Pattern:* a producer batches messages at ≥ 8 in flight; a consumer drains. Throughput, not tail latency, is the goal. *Why matters:* the SHM throughput finding; POSIX named-sem fast path saves a syscall on most operations. *Recommendation:* **POSIX shared memory with named semaphores**. *Examples:* microservice IPC, audio plugin pipeline, co-resident process pair sharing a high-rate event stream.

7. Mostly-idle inter-process notification. *Pattern:* two processes coordinate occasionally; q=1, small messages, latency-insensitive. *Why matters:* q=1 small-message cross-cache is the only common cell where SysV MQ wins on throughput, but the absolute difference is tiny ($\sim 12\%$). The choice is dominated by API ergonomics, not performance. *Recommendation:* **POSIX MQ** for default code-quality reasons (FD integration, simpler cleanup), unless other capability needs intervene.

8. Reader-writer locks across processes. *Pattern:* multiple readers, a single writer; readers must exclude writers and vice-versa, but multiple readers may proceed together. *Why matters:* the canonical implementation uses a counter and a writer-mutex updated atomically together; this is exactly the multi-semaphore atomic case. *Recommendation:* **System V semaphores**, or a purpose-built userspace primitive if portability isn't a constraint.

9. IPC inside an event-driven server (epoll-based). *Pattern:* an event loop awaits sockets, timers, signal sources, and IPC together via a single `epoll_wait`. *Why matters:* POSIX MQ exposes a file descriptor that plugs into `epoll`; SysV MQ does not. *Recommendation:* **POSIX MQ**. The SysV alternative requires `eventfd` bridging which is a known awkwardness. *Examples:* nginx-style worker, custom HTTP servers, async runtimes.

10. Legacy-system interoperability. *Pattern:* the new component must talk to a long-standing service that already uses one IPC family (often System V because of its longer history on Unix). *Why matters:* the choice is fixed by the existing endpoint. *Recommendation:* **Match the existing interface**. The performance considerations in this paper become secondary to interface compatibility.

7.4 Recommendations by developer role and field

The following are defaults a developer in each role should reach for, with notes on when to break the default.

Database, storage, and queueing-system engineers. Default to **POSIX MQ** for ingestion, log streams, and producer-consumer fan-in. Default to **System V semaphores** for transactional locks that span multiple resources. Both defaults are dictated by signature findings of this paper: the fan-in scalability gap and the atomic multi-sem capability.

Real-time, safety-critical, and embedded engineers. Tail latency is the dominant metric. On multi-CCD AMD: pin processes within one CCX whenever possible, and either family works fine. If cross-CCX placement is unavoidable, prefer **System V MQ** for predictable p99.9. Treat the cross-CCX anomaly as architecture-specific: on Intel monolithic dies and on ARM big.LITTLE chips the result will not generalise without additional measurement.

Game-engine and entertainment-systems developers. Inter-process event buses, plugin pipelines, and audio middleware are typically high-rate small-message workloads. Default to **POSIX shared memory with named semaphores** for the batched-throughput advantage. Reach for SysV only when the engine's event-routing primitive is genuinely a kernel-side type-filter (rare).

Web backend and microservice engineers. File-descriptor integration with `epoll` is almost always desired. Default to **POSIX MQ**. Cross-CCX issues matter less because backend latency budgets are usually millisecond-scale, dominated by network and database I/O, not by microsecond IPC tail. Capability cases (atomic multi-sem, `mtype` routing) rarely arise in this domain.

Systems, kernel, and OS engineers. Understand both. Choose per-deployment based on the workload profile rather than family loyalty. Treat the data in this paper as a starting reference point; instrument the actual workload before committing.

Application developers (general-purpose). POSIX as the default. Reach for **System V** only when one of two specific needs arises: (a) atomic multi-resource locking via `semop`; (b) kernel-side message-type filtering via `mtype`. If neither applies, POSIX gives cleaner code, better Unix-idiomatic integration, and better fan-in scaling.

Security-critical and high-assurance developers. System V semaphores simplify correctness reasoning for multi-resource locks. Atomic semantics eliminate an entire class of TOCTOU and partial-acquisition bugs. The performance penalty (one extra syscall trap per opera-

tion) is usually acceptable for the correctness gain, and is dwarfed by the cost of any incident a non-atomic acquisition could cause.

7.5 Common pitfalls

Recurring mistakes that the data in this paper helps avoid:

Pitfall 1: "Always pick the newer one". A developer who reads "POSIX is the modern interface" and chooses POSIX semaphores for multi-resource locking will hand-roll an atomic-acquire primitive that is deadlock-prone and hard to get right. **System V's `semop` solved this in 1983**; do not re-solve it.

Pitfall 2: "Always pick System V because it's older and more powerful". A developer who chooses System V MQ for a high-fan-in log shipper will lose 5.6× throughput at $N = 16$ producers compared to POSIX MQ, with no compensating capability gain.

Pitfall 3: "POSIX MQ is fine for low-jitter cross-CCX". On multi-CCD AMD with cross-CCX placement, POSIX MQ exhibits a p99.9 latency ∼13× worse than SysV. A naive choice of POSIX in a real-time control loop on a Ryzen workstation can violate a 100 μs tail budget every thousand iterations.

Pitfall 4: "Shared memory is always faster than message queues". At q=1 with small messages on cross-cache placement, MQ can match or beat SHM because the SHM path still pays four semaphore operations per message and the kernel queue path is competitive. **Always measure your batching depth** before assuming SHM wins.

Pitfall 5: "System V `semop` is just slow". `semop` traps to the kernel unconditionally, which is more expensive per call than POSIX named-sem's fast path. But that cost buys atomic multi-operation semantics — a property that POSIX semaphores cannot match. The right framing is not "slower" but "more capable, at a fixed per-call cost".

Pitfall 6: Forgetting to clean up SysV resources. SysV IPC objects persist until explicitly removed, even after all processes using them exit. A crashed process leaves a stale `shmid` or `msqid` that consumes system-wide resources counted against `kernel.msgmni`, `kernel.shmmni`, etc. Always handle teardown explicitly (`IPC_RMID` on shutdown, `ipcrm` for orphans). POSIX is friendlier here: `shm_unlink` of a path frees the underlying object, and per-FD limits make exhaustion local rather than global.

Pitfall 7: Cross-stack assumptions about cleanup. POSIX shm objects are visible in `/dev/shm`; SysV in `ipcs` output. Mixing the two without understanding which namespace each lives in confuses both diagnosis and cleanup.

7.6 Decision flow

The following ordered list yields a decision in seven yes/no questions. The first yes ends the search.

1. *Do you need atomic operations on multiple semaphores at once?* If yes, **System V semaphores**. POSIX cannot do this without a hand-rolled atomic primitive.
2. *Do you need kernel-side message routing by an arbitrary type tag (not just priority)?* If yes, **System V MQ** with `mtype`. POSIX MQ supports only priority.
3. *Will many producers (≥4) contend on one consumer queue?* If yes, **POSIX MQ**. SysV MQ collapses super-linearly under contention.
4. *Are you on multi-CCD AMD silicon **and** care about p99.9 tail latency **and** processes may be scheduled across CCX boundaries?* If yes, either pin within one CCX (`same_core` or `same_l3`) and use POSIX, or use **System V MQ** for predictable cross-CCX tails.
5. *Is this a high-rate small-message ring buffer with batching depth ≥8?* If yes, **POSIX SHM with named semaphores**.
6. *Do you need to integrate IPC with a poll/epoll-based event loop?* If yes, **POSIX MQ**. SysV does not expose file descriptors.
7. *Default for new code:* **POSIX MQ**. Cleaner API, automatic per-FD cleanup, integrates with Unix idioms, and the faster choice for the common single-pair p50 case.

We close with one cross-cutting reminder: this decision flow captures only the workload classes our benchmarks measured. For unusual workloads — in particular high-priority preemption schedules, mixed read/write patterns, multi-machine federations, or constraints we did not study — a developer should measure their own workload before committing.

8 Threats to Validity

We organise threats by Wohlin's standard categories.

8.1 Internal validity

The most important internal threat is the **absence of CPU isolation**. We did not pass `isolcpus=` to the kernel, did not fully migrate IRQs off the test cores (modern Linux marks most IRQs as managed, refusing reassignment), and did not disable scheduler ticks. Kernel housekeeping, IRQ handling, and any user-space activity the OS chose to schedule on the test cores therefore co-existed with the benchmarks. p50 medians are robust to this noise (they sit within the unaffected bulk of the distribution), but p99 and p99.9 medians include scheduler interruptions in addition to genuine IPC tail behaviour. We mitigate this in two ways: by using the `performance` CPU governor and disabling Turbo Boost (so that clock-frequency variation is not an additional confound), and by randomising run order so that any drift over the multi-hour suite is distributed across configurations rather than concentrated in one mechanism. The *relative* comparison between POSIX and System V is therefore more robust than the absolute nanosecond numbers.

A secondary internal threat is **cross-stack run-order correlation**: within a stack, run order is randomised, but the entire POSIX block runs before the entire System V block. Long-term drift (thermals, system load) therefore affects the two stacks differently. On a quiet desktop in a single multi-hour session this is unlikely to be a large effect; we note it for completeness.

A third internal threat: the `scalability` benchmark binary uses round-robin pinning across logical CPUs rather than exposing a placement axis. Fan-in and parallel-pairs results therefore

implicitly mix all three placement axes (same_core, same_l3, diff_l3) according to which cores the kernel happens to pick. This is fine for the fan-in scaling result (the lock-contention story is dominated by N, not by placement) but means we cannot isolate placement effects in scalability data the way we can in ping-pong data.

8.2 External validity

We measured one machine. Results are characterised against that machine, not generalised. In particular:

- The cross-CCX tail anomaly is, by construction, **specific to multi-CCD AMD topology**. Single-CCD AMD chips, single-die Intel chips, and ARM big.LITTLE chips would not exhibit the same `diff_l3` placement axis at all. We do not generalise the result.
- No NUMA / cross-socket measurement is performed (single-socket host). NUMA-crossing penalties on multi-socket systems may dominate the behaviour we attribute to cross-CCX in this paper.
- Mitigations are at Ubuntu's defaults. Absolute syscall costs shift materially when mitigations are turned off (KPTI, retpolines, etc.); relative comparison is more robust because both stacks pay the same mitigation tax. We do not produce a paired "mitigations off" baseline.
- Compiler is gcc with `-O2`; we did not explore `-O3`, LTO, PGO, or alternative libcs.

8.3 Construct validity

Latency is measured at the producer-side via `CLOCK_MONOTONIC_RAW`; the inter-process round-trip therefore includes producer wakeup and consumer wakeup as a single number. Throughput is measured around the producer loop only and excludes consumer-side post-processing. Warmup is fixed at 5% of iterations capped at 5000; we did not study sensitivity to warmup length and a larger warmup might further reduce cold-cache effects in p999. We deliberately do *not* report a memory-footprint metric: the available proxy (`VmRSS` delta) is not reliably attributable to IPC kernel memory and would mislead more than inform.

8.4 Conclusion validity

Twenty runs per cell with bootstrap 95% CIs is the minimum sample size we consider acceptable. Cells with overlapping CIs are reported as "no detected difference" rather than "equivalent" — the absence of statistical evidence for a difference is not evidence of no difference. The Mann-Whitney U test is the primary statistical tool because IPC latency distributions are not Gaussian; Welch's t test is reported for completeness. Effect-size reporting via Cliff's δ guards against the trap of small but statistically significant effects in large samples.

9 Future Work

The current study suggests a number of natural follow-ups:

1. **Per-iteration sample dump** (`--dump-samples` flag). Would let us construct full HdrHistogram-style CDFs and characterise the cross-CCX p99.9 tail directly: is it one outlier class, or a

heavier-tailed distribution? Cheap to add to the C benchmarks; primarily a question of disk space.

2. **Asynchronous I/O and `io_uring` comparison**. Modern Linux exposes `io_uring` as a high-throughput asynchronous I/O interface that bypasses much of the syscall path. A direct comparison against POSIX MQ on the same fan-in workload would quantify the residual cost of the syscall-per-operation model even on the futex fast path. Likewise, async-I/O wrappers around both POSIX and System V MQ would test whether the fan-in collapse persists when the synchronous-blocking pattern is replaced.

3. **CPU isolation re-run**. Re-run the suite with `isolcpus=` and IRQ pinning so that p99/p99.9 numbers become defensible as IPC-attributable rather than partly scheduler-attributable. The relative comparison is robust either way; this would close the gap on absolute claims.

4. **Cross-architecture replication**. Repeat the suite on Intel hybrid (P+E cores), Intel server (Xeon), and ARM Neoverse. The futex/spinlock implementation difference is platform-independent in principle but the specific numbers will differ; cross-architecture data would sharply increase external validity.

5. **NUMA / cross-socket study**. The current machine is single-socket. A multi-socket measurement would let us position the `cross_socket` axis relative to `diff_l3`; we expect `cross_socket` to be qualitatively similar but with larger absolute penalties.

6. **Mitigations on/off paired study**. Quantify the mitigation tax separately on POSIX and System V; expected to be roughly symmetric across mechanisms but worth confirming.

7. **Capability-axis benchmarks**. Specifically: the throughput of atomic multi-resource locking using System V `semop` versus a POSIX hand-rolled multi-acquire; the end-to-end dispatch latency of `mtype`-routed messages versus POSIX-priority messages. These would close the comparison's missing half (the capability axis we acknowledge but do not measure here).

8. **SHM fan-in benchmark**. Multi-producer shared-memory ring buffer with proper memory ordering. Deferred in this study because a fair MPSC implementation is itself a research artifact.

9. **Cross-CCX coherency tracing**. Direct instrumentation of the futex wake path and Infinity Fabric coherency events to test the hypothesis advanced in Section 6.2.

10 Conclusion

We have presented a comprehensive comparison of POSIX and System V inter-process communication on Linux, run on a multi-CCD AMD desktop with a statistically rigorous methodology and 10 200 measurements. Three findings stand out:

1. **POSIX MQ scales dramatically better than System V MQ under producer fan-in.** At sixteen contending producers, POSIX delivers approximately 5.6× the throughput of System V. The mechanism is the futex-based wait-queue infrastructure that POSIX MQ inherits from the kernel's modern synchronisation path, contrasted with System V's per-queue spinlock.

2. **POSIX MQ exhibits a reproducible cross-CCX p99.9 latency anomaly** on AMD multi-CCD topology, where it is approximately 13× worse than System V at the diff_l3 placement. This finding is architecture-specific and does not appear at within-CCX placements.

3. **POSIX shared memory is** 1.7–2.5× **faster than System V shared memory at moderate batching depths**, attributable to the futex fast path on POSIX named semaphores versus the unconditional kernel trap on System V `semop`.

At the typical case (p50), the two mechanisms are essentially equivalent for the simple two-process case. The choice between them should therefore be made on tail behaviour, scalability, and capability, not on common-case speed.

The principal deliverable of the work is the developer-facing guidance in Section 7: a capability matrix (§7.1), a performance summary (§7.2), ten use-case archetypes mapping real-world workloads to recommended mechanisms (§7.3), recommendations by developer role and field (§7.4), seven common pitfalls and how to avoid them (§7.5), and a seven-question decision flow (§7.6).

We emphasise what we did not measure: System V's atomic multi-semaphore `semop` and `mtype` message routing are genuine capability advantages with no POSIX equivalent; they are reasons to choose System V for workloads that need them, regardless of timing. Our recommendation is therefore not "always pick the faster one"; it is "identify which capability axes your workload needs, then pick the family whose strengths align with those needs. For new code with no special capability requirements, default to POSIX".

Future work in brief. The most immediate follow-ups are (i) a per-iteration sample dump to characterise the cross-CCX p99.9 anomaly with full distributions, (ii) a paired comparison against `io_uring`-based asynchronous IPC to test whether the fan-in collapse persists once the synchronous-blocking pattern is removed, (iii) a cross-architecture replication on Intel hybrid and ARM Neoverse silicon to test how AMD-specific the cross-CCX behaviour is, and (iv) capability-side benchmarks (atomic multi-semaphore throughput, `mtype` dispatch latency) so that the comparison also covers the axes on which System V leads. A complete list with discussion is in Section 9.

The benchmark suite, raw data, analysis code, and configuration files are available at the project repository for reproduction.

References

[1] Advanced Micro Devices, Inc. AMD "Zen" Architecture Overview. Whitepaper, AMD, 2019.

[2] Advanced Micro Devices, Inc. AMD Infinity Architecture. https://www.amd.com/en/technologies/infinity-architecture, 2024.

[3] Norman Cliff. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin*, 114(3):494–509, 1993.

[4] Ulrich Drepper. Futexes Are Tricky. Technical report, Red Hat Inc., 2011.

[5] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[6] IEEE and The Open Group. IEEE Standard for Information Technology – Portable Operating System Interface (POSIX®). IEEE Std 1003.1-2017, 2017.

[7] Linux man-pages project. clock_gettime(2) – clocks and timers. https://man7.org/linux/man-pages/man2/clock_gettime.2.html, 2024.

[8] Linux man-pages project. futex(7) – fast user-space locking. https://man7.org/linux/man-pages/man7/futex.7.html, 2024.

[9] Linux man-pages project. mq_overview(7) – Overview of POSIX message queues. https://man7.org/linux/man-pages/man7/mq_overview.7.html, 2024.

[10] Linux man-pages project. sched_setaffinity(2) – set a thread's CPU affinity mask. https://man7.org/linux/man-pages/man2/sched_setaffinity.2.html, 2024.

[11] Linux man-pages project. sysvipc(7) – System V interprocess communication mechanisms. https://man7.org/linux/man-pages/man7/sysvipc.7.html, 2024.

[12] Henry B. Mann and Donald R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1):50–60, 1947.

[13] Larry McVoy and Carl Staelin. lmbench: Portable Tools for Performance Analysis. In *Proceedings of the USENIX Annual Technical Conference*, pages 279–294, 1996.

[14] W. Richard Stevens. *UNIX Network Programming, Volume 2: Interprocess Communications*. Prentice Hall, 2 edition, 1999.

[15] Bernard L. Welch. The generalization of Student's problem when several different population variances are involved. *Biometrika*, 34(1/2):28–35, 1947.

A Full system topology

The verbatim output of the topology snapshot captured by `run_suite.sh` into `results/logs/topology.txt`.

```
Generated: 2026-04-28T04:42:01+05:30

=== System ===
Linux ubuntu 6.17.0-22-generic #22~24.04.1-Ubuntu SMP PREEMPT_DYNAMIC
Thu Mar 26 15:25:54 UTC 2 x86_64 x86_64 x86_64 GNU/Linux

=== CPU ===
Architecture:                            x86_64
CPU op-mode(s):                          32-bit, 64-bit
Address sizes:                           43 bits physical, 48 bits virtual
Byte Order:                              Little Endian
CPU(s):                                  12
On-line CPU(s) list:                     0-11
Vendor ID:                               AuthenticAMD
Model name:                              AMD Ryzen 5 3600X 6-Core Processor
CPU family:                              23
Model:                                   113
Thread(s) per core:                      2
Core(s) per socket:                      6
Socket(s):                               1
Stepping:                                0
Frequency boost:                         disabled
CPU(s) scaling MHz:                      99%
CPU max MHz:                             3801.0000
CPU min MHz:                             558.1890
```

```
BogoMIPS:                              7586.26
L1d cache:                             192 KiB (6 instances)
L1i cache:                             192 KiB (6 instances)
L2 cache:                              3 MiB (6 instances)
L3 cache:                              32 MiB (2 instances)
NUMA node(s):                          1
NUMA node0 CPU(s):                     0-11

=== Compiler ===
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0

=== Mitigations ===
gather_data_sampling           Not affected
ghostwrite                     Not affected
indirect_target_selection      Not affected
itlb_multihit                  Not affected
l1tf                           Not affected
mds                            Not affected
meltdown                       Not affected
mmio_stale_data                Not affected
old_microcode                  Not affected
reg_file_data_sampling         Not affected
retbleed                       Mitigation: untrained return thunk;
                               SMT enabled with STIBP protection
spec_rstack_overflow           Mitigation: Safe RET
spec_store_bypass              Mitigation: Speculative Store Bypass
                               disabled via prctl
spectre_v1                     Mitigation: usercopy/swapgs barriers and
                               __user pointer sanitization
spectre_v2                     Mitigation: Retpolines; IBPB: conditional;
                               STIBP: always-on; RSB filling;
                               PBRSB-eIBRS: Not affected; BHI: Not affected
srbds                          Not affected
tsa                            Not affected
tsx_async_abort                Not affected
vmscape                        Mitigation: IBPB before exit to userspace

=== Boost / Governor ===
cpufreq boost: 0
governor: performance

=== Placement pairs ===
Logical CPUs : 12
Sockets      : 1
L3 clusters  : 2
L2 clusters  : 6
same_core    : 0,6
same_l2      : <none>
same_l3      : 0,1
diff_l3      : 0,3
cross_core*  : 0,1   (* = backward compat alias)

=== Run config ===
RUNS=20  PP_ITERS=50000  TP_MSGS=200000  SC_K=10000
RANDOMIZE=1
```

B Master comparison table (selected rows)

The full `master_comparison.csv` contains approximately 600 rows — one per (family, benchmark, placement, msg_size, queue_depth, n_procs, metric) cell. Selected representative rows

appear in Table 10, rounded to three significant figures in the ratio column for compactness. The full table is available in `results/analysis/tables/master_comparison.csv`.

Table 10: Selected headline rows from `master_comparison.csv`. Ratio is sysv/posix; values < 1 favour SysV when the metric is latency, and favour POSIX when the metric is throughput.

Bench	**Cell**	**Metric**	**Ratio**	**CI lo**	**CI hi**	p	**Winner**
fanin	64 B, $N = 2$	MB/s	0.845	0.845	0.866	6.8×10^{-8}	POSIX
fanin	64 B, $N = 4$	MB/s	0.743	0.737	0.752	6.8×10^{-8}	POSIX
fanin	64 B, $N = 8$	MB/s	0.398	0.397	0.402	6.8×10^{-8}	POSIX
fanin	64 B, $N = 16$	MB/s	0.179	0.178	0.180	6.8×10^{-8}	POSIX
pingpong	diff_l3, 8 B	p50	1.011	1.010	1.012	6.7×10^{-6}	POSIX
pingpong	diff_l3, 8 B	p999	0.075	0.075	0.075	6.8×10^{-8}	SysV
pingpong	diff_l3, 64 B	p999	0.080	0.080	0.081	6.6×10^{-5}	SysV
pingpong	diff_l3, 4096 B	p999	0.100	0.100	0.100	9.7×10^{-6}	SysV
pingpong	diff_l3, 8192 B	p999	0.120	0.120	0.120	2.1×10^{-6}	SysV
throughput	same_core, 64 B, q=8	MB/s	0.94	0.93	0.94	6.8×10^{-8}	POSIX
throughput	same_l3, 64 B, q=1	MB/s	1.12	1.10	1.13	6.8×10^{-8}	SysV
throughput	diff_l3, 8 B, q=32	MB/s	1.05	1.05	1.06	6.8×10^{-8}	SysV

C Reproducibility

The benchmark suite is at `stacks/POSIX/` and `stacks/SYSV/` within the project repository. The driver scripts are `scripts/run_suite.sh`, `scripts/run_perf.sh`, and `scripts/run_strace.sh`. Analysis is via `scripts/analyze.py`. To reproduce:

```
sudo -v                 # cache sudo creds for sysctl/ulimit raise
make run                # full timing suite (~2-6 hours)
make perf               # hardware-counter pass (optional, ~30-60 min)
make strace             # syscall summary pass (optional, ~10-20 min)
make analyze            # tables, figures, narrative report
```

Configuration is in `config/defaults.env`: 20 runs per cell, randomised order, 50 000 ping-pong iterations, 200 000 throughput messages, 10 000 scalability messages per producer/pair.